Vulnerability Model and Evaluation of the UEFI Platform Firmware Based on Improved Attack Graphs

Fei Cao
State Key Laboratory of Mathematical Engineering and Advanced Computing
Zhengzhou, China
893374381@qq.com

Qingbao Li
State Key Laboratory of Mathematical Engineering and Advanced Computing
Zhengzhou,China
13653716702@139.com

Zhifeng Chen
State Key Laboratory of Mathematical Engineering and Advanced Computing
Zhengzhou,China
xiaohouzi@163.com

Abstract- Targeted at the situation of rampant attack on UEFI Platform Firmware, this paper sununarizes the UEFI platform firmware framework structure as well as its potential security problems. Then the vulnerability **factors** of UEFI platform firmware are described using the modeling language. **This** paper proposes an improved Attack Graphs model based on Finite State Machine (AGFSM), which can be used to evaluate the reliability of UEFI, as well as calculate the expected exploitation cost and expected attack loss. Finally, based on the AGFSM model, we realize the evaluation of a set of attack strategies for the UEFI platform firmware and verify the rationality of the model and the validity of the evaluation. The research is conducive to the rapid detection of the vulnerability of UEFI Platform Firmware, predict the attack paths and deploy the security **strategy** targeted to safeguard the firmware.

*Keywords-UEFI platform **firmware**, vulnerability, attack path, FSM, CVSS, attack graph*

I. INTRODUCTION

Unified Extensible Firmware Interface (UEFI) provides an abstract programming Interface specification between the operating system and Platform Firmware. In addition, UEFI provides platform initialization and boot services for the operating systeml'l. As a replacement for the traditional BIOS, UEFI has a better extensibility, a more flexible configuration, and a more standard boot process. However UEFI faces more security threats. Kaspersky's 2018 threat prediction safety bulletin mentions more advanced UEFI and BIOS attacksl-l. In fact, the commercial UEFI malware has existed since 2015. For example, the UEFI weapons tool for MAC is also mentioned in the CIA's cyber weapons documentation leaked by WikiLeaks. As the scale of UEFI expands and security problems emerge endlessly, modeling and evaluation of vulnerability will be helpful for the security personnel to analyze the vulnerability relations between the platform components. What's more, it is helpful to build attack scenarios and targeted supplement the UEFI platform firmware security policy.

The contribution of this paper is as follows: (1) We first propose and implement the vulnerability modeling of the UEFI platform firmware, which is an innovative approach to UEFI and BIOS security. (2) We describe the structure composition, the security threat and the attack behavior of the UEFI platform firmware in the modeling and formal language, which provides theoretical support for the scientific definition and the vulnerability evaluation in the future. (3) We measure the reliability of the UEFI platform firmware based on the improved CVSS standard and perceive system vulnerability overall. (4) We quantify the attack path from both the expected cost and the expected loss. Through analyzing and predicting the attack path, we can solve problems with unknown and vague attacks, where targeted security strategies are difficult to deploy.

The following is how the rest of the paper is organized: Section 1 summarizes the related research, with a focus on the vulnerability source of UEFI platform firmware. In Section 2, we define and represent the modeling elements, and propose the AGFSM model and its advantages in view of the state transition and the atomic attack behavior. In the Section 3, the reliability evaluation of the UEFI platform firmware system is realized by improving the CVSS scoring standard. We then evaluate the attack path based on the perspective of cost expectation and loss expectation in Section 4. In Section 5, we build the AGFSM attack graph based on the known UEFI attacks and verify the rationality of the model and the validity of the evaluation in Section 5. Section 6 is the conclusion.

II. RELATED RESEARCH

The UEFI platform firmware mainly includes three parts: hardware, firmware kernel and boot manager. Fig. 1 summarizes the structure of the UEFI platform firmware.

On the basis of understanding the structure of the UEFI platform firmware and referring to the existing attack instances, we summarize the security threats of the UEFI platform firmware in several aspects:

1)Storage Related Threats. The UEFI platform firmware is stored in SPI Flash and UEFI applications are stored in EFI System Partition (ESP), such as bootmgfw.efi. The Option ROM also stores the *PE/COFF* format UEFI image.The SPI

978-1-5386-6565-7/18/$31.00 ©2018 IEEE

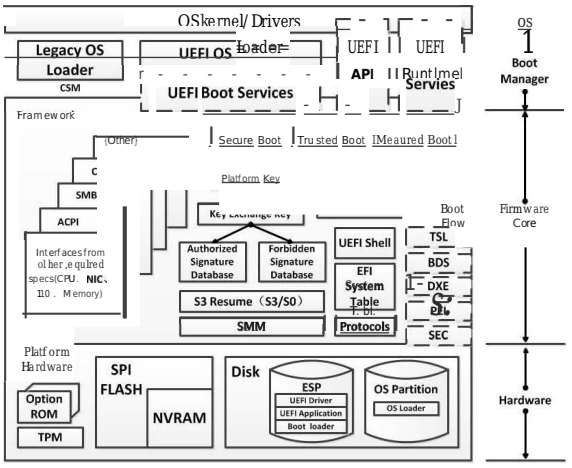

Fig. 1. Architecture of UEFI P1atfonn Firmware

Flash can be easy erased!", the hard disk sector is not protected and the Option Rom can be rewrittenl", These features are security risks.

2)Boot Related Threats. These mainly relate to the UEFI boot process, such as the initialization in the PEl stage, the DXE Dispatcher in the DXE stage and the infection of boot loader in BDS stagel-l, Because of the single defect of trust chain, hijack any stage of the boot process makes the remaining process under threat.

3)Module Expansion Threats. The UEFI specification provides a standardized interface for the motherboard vendor to develop the UEFI extension program and then loads the service into the memory through the Loadlmagei) boot service. With the openness and flexibility of the UEFI platform, malicious third-party applications or drivers may also be loaded, which can be vandalized and hard to detectl'",

4)Programming Related Threats. Intel released the EFI Developer Kit (EDK) project on Tianocore's website, providing open source UEFI implementation code. By analyzing of the Secure Boot code, the attacker can modify the IsSignatureFoundlnDatabaseO function return value to bypass the UEFI image authenticationl'",

5)Protocol Threats. UEFI platform firmware provides the TCP/IP Network Protocol, the Simple File System Protocol, the Block I/O Protocol and the Disk I/O Protocol. These functions which are implemented in the operating System originally can be excuted in the UEFI enviromuent providing a way for an attacker to steal information. Vincent Zimmerlf mentions the remote startup and configuration modification of UEFI.

6)Other Threats. SMM is a CPU execution mode with the highest permissions, and attackers can modify UEFI configuration and hardware chip through SMM vulnerability [9]Besides, the attackers may attack the S3 Resume Boot Script [10], the Secure Boot [11] [12], UEFI update and tamper with the UEFI variablesl'>l, etc.

III. CONSTRUCTING THE AGFSM MODEL

Before constructing the model through an understanding of the UEFI platform firmware and the domain experience, we abstract the structure composition, security threat and target, attack behavior and other modeling elements of UEFI platform firmware into a six-tuple model: *(Mod, Vul, Con, Sec, Sta, Act).* The parameters are as follows:

Mod: The collection of components of the UEFI platform firmware. Any component *h e Mod* in the platform firmware system is represented as *h=(Name, GUID, Val). Name* is the name of the component, *GUID* is the global unique identifier for the component, and *Val* is the asset value.

Vul: The vulnerability of components of the UEFI platform firmware. It refers to a vulnerable point in the UEFI platform firmware or boot process that can be remotely or locally utilized and correspond to one or more of the means of use. Formalization is expressed as *Vul=(GUID, Type, Lim, Act),* each parameter represents the following meaning: *GUID* is the global unique identifier of the component. *Type* is the use of this vulnerability, *R* means remote and *L* means local. *Lim* is the control authority of the component.. *Act* is a set of corresponding atomic attack behaviors.

Con: The connection and trust relationship between the components ofUEFI platform firmware. It is the premise of the vulnerability transfer and the coherence of the attack path. The UEFI platform firmware refers to hardware components such as PCI bus, SPI Flash, NVRAM, Option ROM, hard disk ESP partition, CPU and memory, etc. It also includes software programs such as DXE Core, UEFI driver, boot manager, UEFI Runtime and Boot Service. The connections between the components represent mutual access each other through physical connections while the trust relationships are based on the bootstrap trust chain, which essentially limit the access between components. We use the dual fuzzy relationship to describe: $Connect \subseteq Mod1 \times Mod2$. For example, *Coruh l, h2)* indicates that component hl and h2 have physical connection access or trust transfer relationships.

Sec: The collection of the security attributes for the UEFI platform firmware. This is the safety protection target of the UEFI platform firmware. We learn from the three principles of information security: Integrity, Availability and Confidentiality and use Computational Tree Logic Language to describe these principles. The integrity requirements prevent the intruder from unauthorized "write" for SPI Flash, NVRAM, DXE Driver, boot file, etc. This property is represented as *AG(plvli(write)<root).* The availability requires that any component function still be implemented, including the platform initialization and the boot process. This property is represented as *AG(FunctionCh)=normal).* The confidentiality requirements prevent the intruder from unauthorized "reading" of critical data such as security keys and signature variables. This property is represented as *AG(plvli(read)<root).*

Sta: The collection of the state of the UEFI platform firmware. There are more components involved, and different components specify different states. Therefore, the formal expression is *Sta =(GUID, Req, S.; Snext, w). GUID* is the global unique identifier of the component. *Req* is the condition for the state transfer of the component, which may be the human operation, component life or abnormality, or atomic attack behavior. *Spre* is the set of states before the transfer. *Snext*

is the set of states after the transfer. w stands for the relation of the state transfer. $\omega = S_{pre} \xrightarrow{Req} S_{next}$ represents the process in which the initial state S_{pre} transfers to the result state *Snext* of the component of the UEFI platform firmware, under the premise of the existence of the transfer condition *Req*.

Act: The collection of atomic attack behaviors. Each behavior in this collection is described by a seven-tuple: *a=(ID, Name, VulID, Ss, Sd, Cost, Loss). ID* is the unique number of the behavior. *Name* is the name of the behavior. *VulID* is the vulnerability corresponding to this behavior. If there is no corresponding vulnerability, the value is set to 0. *S,* is the condition state of the behavior (which may contain the initial state of the component, $S_{pre} \subseteq S_s$), *Sd* is the target state of behavior ($S_{next} \subseteq Sd$). *Cost* is the cost of the attack required to complete an atomic attack. *Loss* is the loss caused by an atomic attack. An atomic attack performs an attack step.

We formalize the structural components, security threats and targets, and attack the behavior of the UEFI platform firmware. The next step is to build the vulnerability model to describe the vulnerability utilization process. The vulnerability utilization process represents a series of transitions of the state of the UEFI platform firmware. An atomic attack is a process of using vulnerability as an attacker behavior (performing an attack step). We build the model from two aspects, i.e. the atomic attack behavior produced by vulnerability utilization and the state transition caused by vulnerability utilization.

At present, there are many modeling methods for system vulnerability: attack tree, attack graph, state transition diagram, privilege elevation drawing, permeability map, Petri net, etc. The attack behavior against the UEFI platform firmware is easy to describe while the component types and states are complex. We tend to choose the attack graph as a way of modeling, which is more advantageous to describe attack path, reflecting the causal and temporal relations between atomic attacks. In addition, we also refer to the modeling characteristics of the state transition graph. In order to describe the state migration process in detail, we refer to the Finite State Machine (FSM) model. The model is often used to represent limited states, and the transfer, and behavior between these states. We refer to the definition of the collection of the state *Sta* and the connection and trust relationship between the components *Can* of the UEFI platform firmware at the beginning of this section. The formalization of state migration based on FSM is described below.

Definition 3.1 State transition: The combination of current state S_{pre} and condition *Req* indicates the next state S_{next}, that is, there is a state migration relationship $\underline{\xrightarrow{Req}}$ v represents the state transition between components under a set of atomic attack behavior *Act,* which is expressed as $\upsilon = \omega_1 \xrightarrow{Act} \omega_2$

Definition 3.2 Attack path: An attack path contains the states transition of components and a series of atomic attacks resulting in states transition, which are formally expressed in the following.

From the initial state So $(So \in Sta)$, there is a set of platform firmware state SI,SZ...Sn-I making $(S'' S,_{+}I) E\omega, 0 < i < n-1$. The precondition *Req* of the state transition relationship w is a series of atomic attack collection *Act.* This series of state transitions and atomic attacks ultimately destroys security attributes *Sec,* leaving the sequence *{Random(Sta"Act,)},O < i < n-1* is an attack path.

In conclusion, based on the state collection *Sta* of the UEFI platform firmware, we describe the state transfer process better with FSM. Referring to the definition of the atomic attack collection *Act,* we describe the attack behavior comprehensively, containing the attack cost and the attack loss value. On the whole, the combination of the state transition process and the atomic attack behavior simplifies the attack graph, accurately evaluates the attack path, and clearly shows the vulnerability utilization process of the UEFI platform firmware. *Fig.* 3 is a simple attack path diagram.

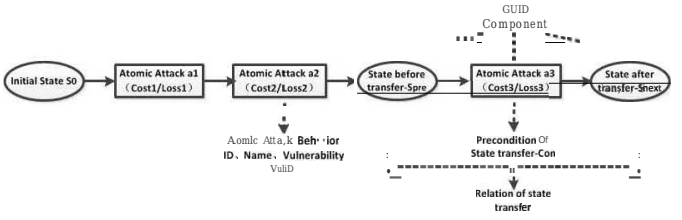

Fig. 2. A simple attack path graph

With the attack path being defined, the combination of multiple attack paths in the UEFI platform firmware forms an attack graph.

Definition 3.3 FSM based attack graph: The binary group *ATFSM = (S,Att)* indicates that an attack graph contains both the state transfer and the atomic attack behavior. As for S = *(Sta, So, Sz V),* which includes the collection of the state of the platform firmware component *Sta,* So ∈ S represents the set of initial state of the UEFI platform firmware, *Sz* ∈ S represents the set of the target state, and *v* is the state transition relationship. As for

Att = (Mod,Modo, Vul, AttNum, Attgoal (min_cos*t,max_loss»),*

where *Modo «Mod* is the starting component of the attacker's attack. *Vul* is the component vulnerability of the UEFI platform firmware, which is the precondition of the attack. *Vul=(GUID, Type, Lim, Act)* includes related components, the utilization way, the control privilege of the component and the set of the atomic attack behaviors involved. *AttNum* refers to the number of steps required to complete an attack. *Attgoal(min_cost, max_loss)* is the target function of the

attacker. *min cost* indicates the minimized attack cost and *max_loss* represents the maximum attack loss caused by the attacker.

We propose an improved attack graph modeling method based on the Finite State Machine proposed in this paper -- Attack Graph model based on FSM (AGFS:M). The model describes and evaluates the vulnerability of the UEFI platform firmware. This modeling approach has two advantages: the diversity of the components of the UEFI platform firmware, the multiplicity of the components involved in the attack path, and the difficulty in unifying state make it difficult for Finite State Machines to describe the vulnerability utilization process of the UEFI platform firmware. However, we can rely on atomic attack definitions to fully describe the attack process. On the other hand, the existence of state points also makes the vulnerability modeling process faster, the model simpler and clearer, and the deployment of the security policies more targeted.

We will further quantify the reliability of the UEFI platform firmware and solve the evaluation and judgment of the optimal attack path. It is helpful to measure the vulnerability of the UEFI platform firmware as a whole, to predict the most likely attack and to assign security strategies to the key nodes.

IV. RELIABILITY MEASUREMENT

The vulnerability assessment is based on the attribute elements of the UEFI platform firmware and the AGFSM attack graph model. In this section, we refer to and improve the Common Vulnerability Scoring System (CVSS)[14], and establish a new scoring mechanism to assess the reliability of the UEFI platform firmware.

A. Concept of Reliability

Reliability refers to the ability of a system or product to complete a predetermined function under specified time or conditions. Referring to the collection of security attributes for the UEFI platform firmware *Sec* defined in Section 2, reliability represents maintaining the integrity, availability, and confidentiality of the UEFI platform firmware, which is expressed as:

AG(plvli(write)<root) and *AG(Function(h)=n01m al)* and *AG(plvli(read)<root).*

B. Improving the CVSS

As an industry open standard, CVSS is often used to assess the severity of the vulnerabilities and to evaluate the attack modelsl"! From the three aspects evaluation Base metric, Temporal metric and Environmental metric, the degree of vulnerability is calculated by formula and mapped to score interval [0.0, 10.0], in order to distinguish the severity of the vulnerability. The base metric responds to the basic properties of the vulnerability directly, which is directly related to how to convert vulnerability utilization and impact degree into attack cost and attack loss. The Temporal metric considers time changes, and the Environmental metric focuses on the vulnerability of the environment and user interaction. Considering the accuracy and the efficiency of the attack path evaluation, the latter two metrics are not included in the attack path evaluation range. The base metric in CVSS includes Access Vector, Access Complexity, Authentication, Confidentiality Impact, Integrity Impact, and Availability Impact. CVSS is highly subjective in the selection of metrics, and the evaluation process is unconvincing and lack of reproducibility. In accordance with specific conditions of UEFI, we add some measurements and normalize the importance ratings. The definition of reliability measurement elements of the UEFI platform firmware based on improved CVSS is shown in *TABLEI.*

In addition to updating the measurement elements and refining severity ratings, We distribute the weight of Influence according to the degree of damage caused by the attack on the confidentiality, integrity, availability, and privilege of the UEFI platform firmware. The reason is that the confidentiality damage is difficult to find and the damage cannot be repaired so it is the most serious. Privilege promotion is a prerequisite for Integrity impact and availability impact. Availability impact is most easily fixed so defined as a general impact factor. Therefore, the weight of Influence is: Availability Impact< Integrity Impact <Privilege Impact < Confidentiality Impact.

(1) Exploitability Score

ES = exAVxACxAU , () is the same as the value in CVSS. The higher the score, the lower the attack cost.

(2) Impact Score

$IS = a\,x[\{(I-CI)x\,\alpha_1\}\times\{(I-II)x\,\alpha_2\}\times\{(1-AI)\times\alpha_3\}\times\{(I-PI)x\,a\,4\}]$, where $\alpha_3 \le \alpha_2 \le a4 \le al$. a is the same as the value of 10.41 in CVSS. The higher the score, the worse the damage.

(3) Protect Score

$PS = \beta\times SB\,x\,SW\,x\,SU\,x\,OP$, Where β represents the influence weight which can be set to 15. The higher the score, the less the protection mechanism.

(4) Base Score

$BaseScore = (0.5x\,PS + 0.3x\,ES + 0.2x\,IS - 1.5)x/$,if $IS = 0$, then $f = 0$, otherwise $f = I.176$. The base score reflects the severity of the threat. The higher the score, the greater the harm or the vulnerability of the system, and the worse the reliability of the UEFI platform firmware.

TABLE!. IMPROVED MEASUREMENTS IN BASE METRIC

	Measurements	Optional Value	Score
1	Confidentiality Impact	NoneiLow/Medium/ High/Critical	0/0.39/0.69/0.89/1.0
2	Integrity Impact	NoneiLow/Medium/ High/Critical	0/0.39/0.69/0.89/1.0
3	Availability Impact	NoneiLow/Medium/ Hiah/Critical	0/0.39/0.69/0.89/1.0
4	Privilege Impact	NoneiLow/Medium/ High/Critical	0/0.39/0.69/0.89/1.0
5	Access Vector	Physical/Local/ LANlInternet	0.2/0.55/0.62/0.85
6	Access Complexity	HighlMiddleiLow	0.44/0.66/0.88
7	Authentication	HighiLow/None	0.27/0.62/0.85
8	Secure Boot	On/Off	0.3/1.0
9	SPIWP	On/Off	0.3/1.0

10	Secure Update	On/Off	0.3/1.0
11	Other Policy	OnlPart/Off	0.3/0.6/1.0

V. EVALUATION OF ATTACK PATH

After measuring the reliability of the UEFI platform firmware, we use the AGFSM attack graph model to evaluate and predict the attack path from an attacker's point of view. In a complete attack path, we consider the cost of the attacker and the damage to the system. In this section, we will define the two variables by using the Expectation of Exploitation Cost(EEC) and the Expectation of Attack Loss (EAL).

A. Cost of Attack path

Definition 4.1 Expectation of Exploitation Cost (EEC) : It indicates the expected value of the cost when the attacker has successfully exploited the vulnerability of the system to achieve its target. The higher the expected value, the higher the attack cost, and the higher the reliability of the system.

For a single unbranched attack path similar to *Fig.* 3, the atomic behavior $a_1, a_2, \cdots, a_n$ is executed successively from the initial state S_0, and finally, the state S_{next} is reached. It is not difficult to understand that the corresponding exploitation cost is expected to be

$$EEC = \sum_{i=1}^{n} EEC_i \tag{1}$$

where EEC_i stands for the cost of atomic behavior a_i, that is, the cost expectation is the sum of the cost of each atomic attack.

B. Loss of Attack path

Definition 4.1 Expectation of Attack Loss (EAL) : It indicates the expected value of the loss when the attacker has successfully exploited the vulnerability of the system to achieve its target. The higher the expected value, the greater the threat of attack and the lower the reliability of the system. Specific measurements include:

(1) Degree of components importance is Importance(h), $\forall h \in [1, NH]$. NH is the number of components. This measure depends on the asset value of the component in the platform firmware, which is the Val of $h = (Name, GUID, Val)$ in the component collection Mod defined in Section 2.

(2) Degree of attack danger is $Danger(a), \forall a \in [1, NA]$, NA is the number of the atomic attacks. In the previous section, the basic measurement group of CVSS is improved and the risk level is determined by two factors: One is the loss caused by an attack in the collection of atomic attack Act, including confidentiality, integrity and availability impact. The second is the permission promotion θ, which is the control of the components acquired after an atomic attack. And θ determines the control of the component assets and the precondition for next attack. So we calculate:

$$Danger(a) = Loss(a) \times \theta_a \tag{2}$$

According to the above measurements, the losses caused to the component are calculated as follows:

$$Damage(a,h) = Importance(h) \times Danger(a), \forall h \in [1, NH], \forall a \in [1, NA]$$

A complete attack path contains the destruction of multiple components, and also contains two random factors, namely the attack component selection and the probability of a successful attack. It can use the Markov Chains to predict attack path selection. The probability of attack success can be derived from historical data or expert experience or can be calculated according to the attack graph model.

An attack path consists of n attack actions $a_1, a_2, \cdots, a_n$. The probability of success of a_i is P_i $i=1,2,\cdots,n$. The probability of success of the attack sequence is

$$P = \prod_{i=1}^{n} P_i \tag{3}$$

An attack path contains many state nodes. We suppose that the probability of success of state S_i to state S_{i+1} is $P_{i,i+1}$. And the probability of success from the initial state S_0 to the state S_i is

$$P_{0,i} = \prod_{j=0}^{i-1} P_{j,j+1} \tag{4}$$

The attack loss of an attack path is the sum of the losses of each component(attack action/state). If the expected loss of S_i is D_i, then the total loss of the attack path is expected to be

$$EAL = \sum_{i=1}^{n} (D_i, P_{0,j}) \tag{5}$$

VI. EXPERIMENTS ANALYSIS

A. Platform firmware configuration and vulnerability

The structure of the UEFI platform firmware is shown in *Fig.* 1. We verify the accuracy of the proposed model and evaluation algorithm according to the general UEFI platform system. The UEFI platform firmware vulnerability list is shown in *TABLE II.* These vulnerabilities cover different UEFI components, which ensures a variety of attack paths. We choose these representative vulnerabilities because of the large scope of harm in the wild.

TABLE II. VULNERABILITIES IN THE UEFI PLATFORM FIRMWARE

List Of Vulnerabilities	Vulnerabilities and Associated modules			
	Module	Vulnerabilities	Seriousness	Description
1	SMM	CVE-2015-0949	M	SMM local code execution vulnerability.
2	SMM	CVE-2017-5721	H	SMM permissions improve vulnerabilities.

3	SPI Flash	CVE-2017-5703	H	Intel CPU vulnerability, allows attackers to write SPI Flash, change or remove BIOS/UEFI firmware,
4	BIOS	CVE-2017-5703	M	BIOS is restarted from the sleep mode, the BIOS CNTL lock protection mechanism is not implemented correctly.
5	Security key	Key leak	H	BIOS source code is compromised, including the AMI BIOS signature key.
6	SPI Flash	Rootkit	M	Zero-day. Remove write protection, execute SMM mode script code, Rootkit
7	S3 Resume Boot Script	PreBoot	H	No restrict access to the EFI S3 startup script, bypass the Secure Boot and refresh the firmware,

B. *Generating and analyzing Attack Graph*

We use the DFS algorithm to generate AGFSM graph automatically. *Fig.* 3 and *TABLEIII* show the description of the attack graph and nodes. The value *cost* and *loss* of the atomic attack node in *TABLEIII* are estimated based on the CVSS measurements (Exploitability Score, Impact Score and Protect Score) in Section 4. The cost and loss of different atomic attacks can be quantified in these measurements. There are two types of node in the attack graph: the square node represents an atomic attack, and the elliptical node represents the component or system state of the DEFI platform firmware.

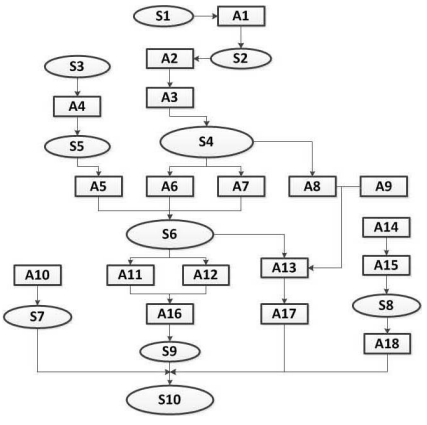

Fig. 3. AGFSM Attack Graph

Improving the attack graph based on the Finite State Machine, we can see AGFSM model contains only the attacker location, vulnerabilities and attack targets and status of the DEFI platform firmware. This simple model can also fully describe the attack behavior. According to *Formula* (5), the cost expectation of attack path is obtained. According to *Formula (10),* the loss expectation of attack path is obtained. We have the following results as *TABLEIV.*

The optimal attack path needs to take into account both the minimum attack cost and the maximum attack loss, which is the optimal solution of $Att_Path = \{Min(EEC), Max(EAL)\}$. This is a constrained binary variable optimization problem. We calculate the optimal attack path 5. The attack exploitation cost is the lowest, that is, the attack is easier to implement, and more components are involved with the highest expected loss. At the same time, many paths include the state S^4 and S^6. We have concluded that it is very important for write protection status of SPI Flash and switch status of the Secure Boot in the DEFI platform firmware security protection. Meanwhile, when the Secure boot is turned on and off, the Base Score calculated by the formula in Section 4.2 is quite different. The actual attack case also proves this conclusion. What's more, with the AGFSM model and evaluation approach, we can also find out more components that need to be targeted for deployment security policies or special attention to the DEFI platform firmware state.

VII. CONCLUSION

This paper tackles an important cyber security problem, i.e., modeling the attack surface of the DEFI platform firmware and performing a quantitative security analysis. We devise an attack graph based on modeling of the vulnerabilities in the DEFI firmware framework and propose two metrics to assess the impact and the cost of the attacks. They use the scores associated with various DEFI attack vectors to infer the attack paths which the attackers would likely to pursue. It is an innovative approach to DEFI and BIOS security.

However, in this paper, the quantity of the relative parameters of the attack path is used to predict the attack, which lacks the consideration of the attacker's intention and the randomness of the attack. In addition, it is difficult to realize automatic modeling, which makes the analysis of DEFI vulnerability less efficient. DEFI technology continues to evolve and improve, with a combination of hardware and network support security policies, such as TPM chips and Measured Boot. At the same time, new attack strategies and high-risk vulnerabilities emerge endlessly. Therefore, the vulnerability modeling and security evaluation of the DEFI platform firmware should be a continuous research process. We look forward to more research and progress in this area.

TABLE III. ATTACK GRAPH DESCRIPTION

State Node	S1	S2	S3	S4	S5
description	SMM vulnerability exits	SMM state=true	Infected BIOS Image	SPI Flash Write Protectiorr-O	Infected code exits In SPI Flash
State Node	S6	S7	S8	S9	SIO
description	Secure Boot bypass	Extra Runtime Services exits	UEFI Shell	Bootkit	OS CORE hijacked
Atomic Attack	A1	A2	A3	A4	A5	A6

Node						
description	SMI Interrupt	Modify BIOS LOCK ENABLE(BLE)=O	Modify SPI Write Protection (WP) =0	Update BIOS ROM	Modify IsSignatureFoundInDatabaseO return EFI SUCCESS	Modify "on/off' of Authenticated Variable in NVRAM
attribute	Cost=l.O Loss=O	Cost=3.0 Loss=5.0	Cost=1.5 Loss=7.0	Cost=6.0 Loss=O	Cost=3.0 Loss=6.0	Cost=2.5 Loss=6.0
Atomic Attack Node	A7	A8	A9	AI0	A11	A12
description	Hashing and Add signature of UEFI Image in DB in NVRAM	Modify path of DXE driver in NVRAM	Modify DXE Dispatcher	ModifyUEFI protocol OnstallProtocolInterface)	Replace Boot Manager (bootmgfw.efi/boot x64.efi) in EFI Svstem Partition	Replace as Loader (winload.efi/winload.exe) in as Partition
attribute	Cost=4.0 Loss=6.0	Cost=3.0 Loss=6.0	Cost=4.0 Loss=4.0	Cost=8.0 Loss=4.0	Cost=1.5 Loss=4.5	Cost=1.0 Loss=4.5
Atomic Attack Node	A13	A14	A15	A16	A17	A18
description	Program DXE Driver	Enter UEFI Shell in high privilege by interrupting boot order	Program UEFI Application	Hook Ntoskml.exe and bypass verifying system	Install UEFI Driver(Driver Binding Protocol: Supportt) StartOStopO)	Install UEFI Application
attribute	Cost=2.0 Loss=O	Cost=8.0 Loss=3.0	Cost=2.0 Loss=O	Cost=3.0 Loss=6.0	Cost=2.5 Loss=2.0	Cost=2.5 Loss=2.0

TABLE IV. EVALUATION OF ATTACK PATHS

	Attack path	EEC	EAL
1	$83 \to A4 \to 85 \to 86 \to \mathrm{Al}\,1 \to \mathrm{A}16 \to 89 \to 810$	10.5	10.5
2	$S3 \to A4 \to S5 \to S6 \to A12 \to A16 \to S9 \to S10$	10.0	10.5
3	$A10 \to S7 \to S10$	8.0	4.0
4	$81 \to \mathrm{Al} \to A2 \to A3 \to 84 \to A6 \to 86 \to \mathrm{Al}\,1 \to \mathrm{A}16 \to 89 \to 810$	12.0	28.5
5	$81 \to A1 \to A2 \to A3 \to S4 \to A6 \to S6 \to A12 \to A16 \to S9 \to S10$	11.5	28.5
6	$81 \to A1 \to A2 \to A3 \to S4 \to A7 \to S6 \to A11 \to A16 \to S9 \to S10$	13.5	28.5
7	$81 \to A1 \to A2 \to A3 \to S4 \to A7 \to S6 \to A12 \to A16 \to S9 \to S10$	13.0	28.5
8	$A14 \to A15 \to S8 \to A18 \to S10$	12.5	5.0
9	$81 \to \mathrm{Al} \to A2 \to A3 \to S4 \to A6 and A8 \to \mathrm{S6}\ \mathrm{and}\ (\mathrm{A8and}A9) \to A13 \to \mathrm{A}17 \to S10$	29.5	36
10	$81 \to \mathrm{Al} \to A2 \to A3 \to 84 \to A7 and A8 \to \mathrm{S6}\ \mathrm{and}\ (\mathrm{A8andA9}) \to \mathrm{A}13 \to \mathrm{Al}\,7 \to S10$	34	36

ACKNOWLEDGMENT

Thanks are due to Chen for assistance with the experiments and to Li for valuable discussion. Thanks to Intel's tool-Chipsec, which plays an important role in UEFI security detection. Last, we thank the anonymous reviewers for their comments to improve the quality of the paper.

REFERENCES

[1] Unified Extensible Firmware Interface Specification.Version2.7. WIVIV.uefi.org/sites/default/files/resources/ UEFI_Spec_2_7.pdf

[2] Juan Andrés Guerrero-Saade, Costin Raiu, Kurt Baumgartner.Kaspersky Security Bulletin: Threat Predictions for 2018 Advanced Persistent Threats in 2018. 2017.https://securelisLcom/ksb-threat-predictions-for-2018/83169/

[3] Kallenberg C, Wojtczuk R. Speed racer: Exploiting an Intel flash protection race condition[J]. Bromium Labs (January 6. 2015) URL: https://bromiumlabs. files. wordpress. com/2015/0l/speed_racer _whitepaper. pdf (data obrashcheniya: 07.01. 2016), 2015.

[4] Zimmer V, Kumar M, Natu M. System and method to secure boot both UEFI and legacy option ROM's with common policy engine[J]. 2014.

[5] S. Kaczmarek. UEFI and Dreamboot. In Hack in the Box, June 2013

[6] Intel.Signing UEFI Applications and Dnvers for UEFI Secure Boot.2013 https://finnware.intel.com/

[7] Rosenbaum L, Zimmer V. A Tour Beyond BIOS into UEFI Secure Boot[J].2012.

[8] Nystrom M, Nicoles M, Zimmer V. UEFI Networking and Pre-OS Security[J]. Intel Technology Journal, 2011,15(1):80-1-1.

[9] Embleton S, Sparks S, Zou C. SMM rootkits: a new breed of as independent malware[J]. Security & Communication Networks, 2013, 6(12):1590-1605.

[10] Bazhaniuk O, Bulygin Y, Furtak A, et al. Attacking and Defending BIOS in 2015[C]//ReCon Conference, http://www. intelsecurity. com/advanced-threat-research/content/ AttackingAndDefending BIOS-RECon2015. pdf.

[11] SlidesBulygin Y, Furtak A, Bazhaniuk O. A tale of one software bypass of Windows 8 Secure Boot[J]. Black Hat USA, 2013.

[12] Jacob N, Heyszl J, Zankl A, et al. How to Break Secure Boot on FPGA SoCs Through Malicious Hardware[J]. 2017.

[13] Kallenberg C, Butterworth J, Kovah X, et al. Defeating signed BIOS enforcement[J]. EkoParty, Buenos Aires, 2013.

[14] Mell P, Scarfone K, Romanosky S. Common Vulnerability Scoring System (CVSS).http://WIVIV.first.org/cvss/cvss-guide. htm12011.

[15] Pooisappasit N, Dewri R, Ray 1. Dynamic .CVSS-based security risk management using bayesian attack graphs. Dependable and Secure Computing, IEEE Transactions on, 2012, 9(1):61-74 .